

Gasverbrauch-Statistik in ioBroker

Tages-, Wochen- und Monatswerte sowie Durchfluss aus dem Zählerstand des HmIP-ESI-GAS

Datei	gasverbrauch_statistik.js
Version	2.1
Stand	21.09.2026
Adapter	javascript (Typ Javascript/ECMAScript)
Erstellt für	ssbingo
Benötigt für	ioBroker.vis-2-widgets-wolf (Gaszähler-Widget)

1. Ausgangslage

Der **HmIP-ESI-GAS** zeigt in der CCU-WebUI vier Statistikwerte an – *Heute, Gestern, Vergangene 7 Tage, Vergangene 30 Tage*. Diese Werte sind **keine Gerätedatenpunkte**: Sie werden CCU-intern aus den gespeicherten Zählerständen berechnet und liegen als Metadaten vor. Über `hm-rpc` kommen sie deshalb grundsätzlich nicht in ioBroker an.

In ioBroker verfügbar sind:

Quelle	Datenpunkt	Inhalt
hm-rpc	GAS_FLOW , GAS_FLOW_STATUS	aktueller Gasfluss in m³/h
hm-rpc	GAS_VOLUME , GAS_VOLUME_STATUS	Volumen in m³ – kein Gesamtzählerstand
hm-rega	svEnergyCounter...<Seriennummer>	Zählerstand – versteckte Systemvariable

Der Zählerstand ist monoton steigend und damit die einzige Größe, die man wirklich braucht: Alles andere lässt sich daraus ableiten. Genau das macht dieses Skript – unabhängig von CCU-Firmware-Internia, die HomeMatic jederzeit ändern kann.

Hinweis zur versteckten Systemvariablen: Damit `svEnergyCounter...` in ioBroker erscheint, muss im Adapter **hm-rega** unter *Synchronisieren* die Option für **nicht sichtbare Variablen** aktiviert werden. Danach die Instanz neu starten und im Objektbaum nach der Seriennummer des Geräts filtern.

2. Installation

1. Im Adapter **javascript** ein neues Skript vom Typ *Javascript/ECMAScript* anlegen.
2. Inhalt von `gasverbrauch_statistik.js` einfügen.

3. Im Konfigurationsblock oben mindestens `SRC` anpassen (siehe Abschnitt 3).
4. Skript starten und das Log prüfen – die States werden automatisch angelegt.

Voraussetzungen:

- Adapter `javascript` (JavaScript-Engine, Node ≥ 18)
- Objektbaum `0_userdata.0` (Standard; anderer Pfad über `PFAD` möglich)
- optional: ein History-Adapter für die einmalige Vorbefüllung (Abschnitt 5)

3. Konfiguration

Alle Einstellungen stehen im Kopf des Skripts.

Konstante	Default	Bedeutung
<code>SRC</code>	<code>'hm-rega.0.12345'</code>	Pflichtangabe. State mit dem Zählerstand in m ³ – die Rega-Systemvariable <code>svEnergyCounter...</code> (hm-rpc liefert für den ESI-GAS keinen Gesamtzählerstand)
<code>PFAD</code>	<code>'0_userdata.0.Gas'</code>	Zielordner für die erzeugten States
<code>INKL_HEUTE</code>	<code>false</code>	<code>false</code> = 7-/30-Tage-Summe zählt nur abgeschlossene Tage (Verhalten der CCU). <code>true</code> = laufender Tag wird mitgezählt
<code>TAGE_HISTORIE</code>	<code>70</code>	Anzahl vorgehaltener Tageswerte. Für <i>Letzter Monat</i> sind mindestens 62 nötig
<code>NK</code>	<code>3</code>	Nachkommastellen der Verbrauchswerte
<code>HISTORY_INSTANCE</code>	<code>'influxdb.0'</code>	Instanz für die Vorbefüllung. <code>''</code> schaltet die Funktion ab. Auch <code>'history.0'</code> oder <code>'sql.0'</code>
<code>BACKFILL_BEIM_START</code>	<code>true</code>	Beim allerersten Start automatisch aus der History vorbefüllen
<code>DURCHFLUSS_AKTIV</code>	<code>true</code>	Durchfluss aus der Zähleränderung berechnen (Abschnitt 7)
<code>FENSTER_MIN</code>	<code>10</code>	Glättungsfenster in Minuten. Größer = ruhiger, aber träger
<code>NULL_NACH_MIN</code>	<code>10</code>	Ohne Zähleränderung für so viele Minuten wird der Durchfluss 0
<code>LUECKE_MIN</code>	<code>7</code>	Abstand zweier Zähleränderungen, ab dem eine Pause angenommen wird ($\leq$ <code>NULL_NACH_MIN</code>)
<code>ANLAUF_MIN</code>	<code>3</code>	Angenommene Dauer der ersten Änderung nach einer Pause ($\approx$ Sendeintervall des ESI)
<code>MAX_DURCHFLUSS</code>	<code>10</code>	Plausibilitätsgrenze in m ³ /h; höhere Werte werden begrenzt und protokolliert
<code>DEBUG</code>	<code>false</code>	Zusätzliche Log-Ausgaben bei jeder Berechnung

4. Erzeugte States

Alle unterhalb von `PFAD` (Default `0_userdata.0.Gas`):

State	Typ	Rolle	Inhalt
<code>Zaehlerstand</code>	number	<code>value.volume</code>	gespiegelter Zählerstand in m³
<code>Heute</code>	number	<code>value</code>	Verbrauch seit Mitternacht
<code>Gestern</code>	number	<code>value</code>	Verbrauch des Vortags
<code>Letzte7Tage</code>	number	<code>value</code>	Summe der letzten 7 Tage
<code>Letzte30Tage</code>	number	<code>value</code>	Summe der letzten 30 Tage
<code>DieserMonat</code>	number	<code>value</code>	Summe seit Monatsbeginn inkl. heute
<code>LetzterMonat</code>	number	<code>value</code>	Summe des Vormonats
<code>Basis</code>	number	<code>value.volume</code>	Zählerstand um Mitternacht (intern)
<code>BasisDatum</code>	string	<code>date</code>	Datum dieser Basis, <code>YYYY-MM-DD</code> (intern)
<code>Historie</code>	string	<code>json</code>	Tageswerte als JSON-Ringpuffer (intern)
<code>Backfill</code>	boolean	<code>button</code>	löst die Vorbefüllung aus (Abschnitt 5)
<code>Durchfluss</code>	number	<code>value</code>	berechneter Durchfluss in m³/h (Abschnitt 7)
<code>VerbrauchAktiv</code>	boolean	<code>indicator</code>	<code>true</code> , solange Gas fließt
<code>ZaehlerLetzteAenderung</code>	number	<code>value.time</code>	Zeitstempel der letzten Zähleränderung (ms)

Die drei internen States nicht von Hand ändern – sie bilden den Zustand ab, aus dem alle anderen Werte berechnet werden.

Format der Historie:

```
[{"d": "2026-09-18", "v": 1.3}, {"d": "2026-09-19", "v": 1.6}]
```

Ein Eintrag je **abgeschlossenem** Tag. Der laufende Tag steht nicht darin, sondern ergibt sich aus `Zaehlerstand – Basis`.

5. Vorbefüllung aus dem History-Adapter

Ohne Vorbefüllung sind die 7-/30-Tage-Werte erst nach 7 bzw. 30 Tagen Laufzeit vollständig. Wird der Zählerstand bereits in `influxdb`, `history` oder `sql` aufgezeichnet, baut das Skript die Historie daraus in einem Rutsch auf.

Automatisch: beim allerersten Start, sofern `HISTORY_INSTANCE` gesetzt und `BACKFILL_BEIM_START = true` ist.

Manuell: den State `<PFAD>.Backfill` auf `true` setzen (Objektbaum oder VIS-Button). Die Historie wird dabei komplett neu aufgebaut.

Verfahren: Für jede Tagesgrenze (00:00 Uhr) wird der Zählerstand per `getHistory` mit `aggregate: 'max'` über ein Fenster davor abgefragt – beim monoton steigenden Zähler ist das Maximum des Fensters genau der Stand zur Tagesgrenze. Der Tagesverbrauch ist die Differenz zweier aufeinanderfolgender Tagesgrenzen. Pro Tagesgrenze fällt nur eine Abfrage mit einem Rückgabewert an; auch bei minütlichem Logging bleibt die Last gering. Liefert das erste Fenster (2 Tage) nichts, wird auf 7 und dann 30 Tage erweitert.

Tage, für die keine History-Daten vorliegen, werden ausgelassen statt mit falschen Werten gefüllt; das Log nennt die Anzahl. Liefert die History gar nichts, bricht die Vorbefüllung mit einer Warnung ab und der laufende Betrieb beginnt bei null – ein typischer Grund ist, dass der Quell-State im History-Adapter nicht zur Aufzeichnung aktiviert ist.

6. Funktionsweise im laufenden Betrieb



Auslöser der Berechnung

Auslöser	Zweck
Änderung von <code>SRC</code>	laufende Aktualisierung von <i>Heute</i>
<code>schedule('1 0 * * *')</code>	Tagesabschluss um 00:01 Uhr
<code>schedule('5 * * * *')</code>	stündliches Sicherheitsnetz
<code>schedule('* * * * *')</code>	minütliche Prüfung: Durchfluss auf 0, wenn der Zähler steht
<code>Backfill = true</code>	Vorbefüllung aus der History

Überlappende Läufe sind durch eine Sperre ausgeschlossen – wichtig, weil die Vorbefüllung länger dauern kann als das Intervall des Zählers. Der Durchfluss wird außerhalb dieser Sperre sofort bei jeder Zähleränderung aktualisiert.

Robustheit

- **Verpasster Mitternachtslauf:** Der Tagesabschluss hängt nicht allein am Cron. Bei jeder Zähleränderung wird geprüft, ob sich das Datum gegenüber `BasisDatum` geändert hat, und der Abschluss gegebenenfalls nachgeholt.
- **Ausfalltage:** War ioBroker mehrere Tage aus, werden die fehlenden Tage mit `0` aufgefüllt, damit die Reihe nicht verrutscht. Der in dieser Zeit angefallene Verbrauch wird dem ersten Tag

der Lücke zugeschlagen – die Summen über 7 und 30 Tage bleiben dadurch korrekt, die einzelnen Tageswerte innerhalb der Lücke nicht.

- **Zählerreset / Zählerwechsel:** Sinkt der Zählerstand, wird die Basis zurückgesetzt und der betroffene Tag mit `0` gewertet, statt negative Werte zu erzeugen. Eine Warnung landet im Log.
- **Ungültige Werte:** `null`, `undefined` und nicht-numerische Werte werden ignoriert, nicht als 0 verarbeitet.

7. Berechneter Durchfluss

Warum nicht einfach `GAS_FLOW` ? Der ESI zählt nur Impulse. Solange Impulse kommen, sendet er im Rhythmus $(A + 1) \times C$ (Abschnitt 8). Hört der Verbrauch auf, kommen keine Impulse mehr – das Gerät hält seinen Zustand für *unverändert* und meldet den Durchfluss 0 erst mit der nächsten unveränderten zyklischen Meldung, mit Standardwerten nach 1,4 bis 2,2 Stunden. Bis dahin steht in `GAS_FLOW` der letzte Wert, obwohl längst kein Gas mehr fließt.

Das Skript rechnet den Durchfluss deshalb selbst aus dem Zählerstand und setzt ihn nach `NULL_NACH_MIN` Minuten ohne Zähleränderung zuverlässig auf 0.

Verfahren

- Jede Zähleränderung ist eine Probe (*Zeit, Stand*).
- **Normalfall:** Durchfluss = $\Delta \text{Stand} / \Delta \text{Zeit}$ zwischen der neuesten Probe und einem Anker am Beginn des Glättungsfensters (`FENSTER_MIN`).
- **Anlauf nach Pause:** Liegt die vorige Änderung länger als `LUECKE_MIN` zurück, enthielte ΔZeit die ganze Pause und der Wert liefe langsam hoch. Stattdessen wird angenommen, dass der Verbrauch innerhalb von `ANLAUF_MIN` angefallen ist – das entspricht einem Sendeintervall des ESI.
- **Stillstand:** Eine minütliche Prüfung setzt 0, sobald `NULL_NACH_MIN` keine Änderung kam.
- **Zählerreset / Sprung:** Sinkt der Stand, beginnt die Berechnung neu (Wert 0). Werte über `MAX_DURCHFLUSS` werden begrenzt und im Log gemeldet.
- **Neustart:** Nach einem Skript-Neustart steht der Durchfluss auf 0, bis die nächste Änderung kommt; der letzte Stand wird aus dem Quell-State übernommen.

Testergebnis (Sandbox mit simulierter Uhr, ESI mit $A = 0$):

Szenario	Eingabe	Ergebnis
Brenner läuft	+0,06 m³ alle 3 min (1,2 m³/h)	1,2 m³/h ab der ersten Meldung
Brenner aus	keine Änderung	nach 9 min noch 1,2 – nach 11 min 0
Warmwasser nach 40 min Pause	+0,03 m³ alle 3 min (0,6 m³/h)	0,6 m³/h sofort
Kleinlast	+0,01 m³ alle 6 min (0,1 m³/h)	0,1 m³/h (Übergang über ein Fenster)
Unplausibler Sprung	+5 m³ in 1 min	auf 10 m³/h begrenzt, Warnung im Log
Zählerreset	Stand fällt auf 0,5	0 m³/h, Neubeginn

Abstimmung der Parameter

Wunsch	Einstellung
0 schneller anzeigen	NULL_NACH_MIN kleiner – aber größer als der Impulsabstand bei kleinster Last lassen, sonst springt die Anzeige bei Kleinlast zwischen 0 und Wert
ruhigere Anzeige	FENSTER_MIN größer
CCU mit Standardwerten (A = 1)	ANLAUF_MIN = 6 , LUECKE_MIN = 10 , NULL_NACH_MIN = 15

Die Genauigkeit ist durch die Impulswertigkeit begrenzt: Bei 0,01 m³/Impuls und 0,1 m³/h kommt nur alle 6 Minuten ein Impuls – feiner lässt sich kleine Last grundsätzlich nicht auflösen.

Aktualität der Quelle: Der Zählerstand kommt als Rega-Systemvariable über hm-rega. Änderungen erreichen ioBroker daher erst mit dem nächsten Abfragezyklus der hm-rega-Instanz (üblich 30 s). Gegenüber dem Sendeintervall des ESI von 2–3 Minuten ist das unkritisch; für sofortige Übernahme siehe *Polling-Trigger* in Abschnitt 8.

8. Empfohlene CCU-Einstellungen

Der Durchfluss im Skript kann nur so aktuell sein wie die Zählerstände, die der ESI liefert. Die Sendehäufigkeit von Homematic-IP-Geräten ergibt sich aus drei Werten:

	Parameter (WebUI)	Standard
A	Anzahl der auszulassenden Statusmeldungen	1
B	Anzahl der auszulassenden, unveränderten Statusmeldungen	20
C	Grundintervall (vom Hersteller fest)	ca. 120–184 s

- bei Verbrauch (Stand ändert sich): Meldung alle **(A + 1) × C**
- ohne Verbrauch (Stand unverändert): Meldung alle **(A + 1) × (B + 1) × C**

Variante	A	B	Meldung bei Verbrauch	ohne Verbrauch
Empfehlung	0	5	alle 2–3 min	ca. 12–18 min
maximal aktuell	0	0	alle 2–3 min	alle 2–3 min
Standard	1	20	alle 4–6 min	ca. 1,4–2,2 h

Einstellung in der CCU: *Einstellungen* → *Geräte* → *HmIP-ESI* → *Einstellen*, bei allen Kanälen, die die Parameter anbieten. **„Zyklische Statusmeldung“ muss aktiviert sein** – A und B regeln nur deren Häufigkeit; ohne sie meldet das Gerät nur bei Änderung, und die Erreichbarkeitsüberwachung der CCU verliert ihre Grundlage.

Nach dem Speichern steht das Gerät auf *Konfiguration ausstehend*. Ein kurzer Druck auf die Systemtaste des ESI überträgt die Konfiguration sofort.

hm-rega: Abfrageintervall und Polling-Trigger

hm-rpc liefert für den ESI-GAS keinen Gesamtzählerstand; die Quelle ist die Rega-Systemvariable `svEnergyCounter...`. Systemvariablen werden von hm-rega nicht ereignisbasiert gemeldet, sondern zyklisch abgefragt.

Einstellung (hm-rega)	Empfehlung
Polling-Intervall	≤ 30 s – deutlich kürzer als das Sendeintervall des ESI
Polling-Trigger (optional)	virtuelle Taste, z. B. <code>BidCoS-RF.50.PRESS_SHORT</code> (ID aus dem Objektbaum kopieren)

Mit Polling-Trigger holt hm-rega die Variablen sofort ab, sobald die virtuelle Taste gedrückt wird. Dazu in der CCU ein Programm anlegen:

- **Bedingung:** Gerät HmIP-ESI, Kanal 2 (Zählerstand) *bei Aktualisierung*
- **Aktivität:** Zentrale, virtuelle Taste 50 → *Tastendruck kurz*

Die Taste darf von keinem anderen Programm verwendet werden. Bei 30 s Polling ist der Trigger für die Statistik nicht nötig; er verbessert nur die zeitliche Genauigkeit des Durchflusses.

Kürzere Intervalle kosten Batterielaufzeit (Herstellerangabe > 7 Jahre gilt für die Standardwerte) und erzeugen mehr Quittierungen durch die Zentrale.

9. Bekannte Grenzen

- **Tagesgrenze.** Die Basis wird mit dem zuletzt gemeldeten Zählerstand gesetzt. Meldet der ESI zuletzt um 23:40 Uhr, zählt der Verbrauch von 23:40 bis 00:00 Uhr zum Folgetag. Bei den üblichen Melderaten liegt der Fehler im Bereich weniger Hundertstel m³ und gleicht sich über die Tage aus.
- **Rollierende Fenster.** *Letzte 7/30 Tage* summieren die letzten 7 bzw. 30 abgeschlossenen Tage. Die CCU rechnet gleichartig, kann bei einer Neuanlage aber eine längere Vorgeschichte haben – kleine Abweichungen zur WebUI sind in den ersten Wochen normal.
- **Sommer-/Winterzeit.** Alle Datumsgrenzen laufen über lokale Zeit; der 25-Stunden- und der 23-Stunden-Tag werden korrekt als ein Tag gewertet.
- **Keine Rekonstruktion ohne History.** Ohne History-Adapter beginnt die Statistik bei null; die CCU-Werte lassen sich nicht rückwirkend importieren.

10. Fehlersuche

Symptom	Ursache / Abhilfe
Log: Quell-State "..." existiert nicht	SRC falsch. Objekt-ID im Objektbaum kopieren

Symptom	Ursache / Abhilfe
Alle Werte bleiben 0	Quell-State liefert keinen Zahlenwert – Typ prüfen (<code>number</code> , nicht <code>string</code>)
Log: <code>History</code> lieferte keine verwertbaren Daten	Quell-State ist im History-Adapter nicht zur Aufzeichnung aktiviert, oder <code>HISTORY_INSTANCE</code> stimmt nicht
Gestern bleibt 0	Noch kein Tagesabschluss gelaufen – erst am Folgetag gefüllt (oder Vorbefüllung nutzen)
Werte weichen von der CCU ab	<code>INKL_HEUTE</code> prüfen; die CCU zählt den laufenden Tag in den 7-/30-Tage-Werten nicht mit
Log: <code>Zählerstand</code> ist gesunken	Zählerwechsel oder Reset der Rega-Variablen – einmalig normal, wiederholt ein Hinweis auf eine instabile Quelle
Historie soll neu aufgebaut werden	<code>Backfill</code> auf <code>true</code> setzen, oder <code>Historie</code> auf <code>[]</code> setzen und Skript neu starten
Durchfluss springt bei Kleinlast zwischen 0 und Wert	<code>NULL_NACH_MIN</code> und <code>LUECKE_MIN</code> zu klein für den Impulsabstand – erhöhen
Durchfluss reagiert träge	<code>FENSTER_MIN</code> verkleinern; CCU-Einstellung A = 0 prüfen (Abschnitt 8)
Durchfluss nach Brennerstart zu niedrig	<code>ANLAUF_MIN</code> an das tatsächliche Sendeintervall anpassen
Log: <code>Durchfluss ... unplausibel</code>	Sprung im Zählerstand (z. B. Offset nachgetragen) – einmalig normal

Für detaillierte Ausgaben `DEBUG = true` setzen; das Skript protokolliert dann bei jeder Berechnung Zählerstand und alle abgeleiteten Werte.

11. Verwendung in VIS-2

Die States sind mit Rolle und Einheit versehen und lassen sich direkt binden:

```
{0_userdata.0.Gas.Heute}           m³
{0_userdata.0.Gas.Gestern}         m³
{0_userdata.0.Gas.Letzte7Tage}     m³
{0_userdata.0.Gas.Letzte30Tage}    m³
{0_userdata.0.Gas.Durchfluss}      m³/h
{0_userdata.0.Gas.VerbrauchAktiv}   true/false – z. B. Flammensymbol ein/aus
```

Für ein Balkendiagramm der letzten Tage eignet sich der State `Historie` (`role: json`) als Datenquelle – er enthält Datum und Verbrauch je Tag bereits in der Struktur, die die meisten Chart-Widgets erwarten.

12. Quellen

- HmIP-ESI Gas – nicht alle Werte im ioBroker: <https://forum.iobroker.net/topic/82859/hmip-esi-gas-nicht-alle-werte-im-iobroker>
- HmIP-ESI (ioBroker-Forum): <https://forum.iobroker.net/topic/73449/hmip-esi>
- HowTo – erweiterte CCU3/RM Verbrauchszähler in eigene SysVars schreiben: <https://homematic-forum.de/forum/viewtopic.php?t=82352>
- HmIP-ESI Datenpunkte für historische Daten: <https://homematic-forum.de/forum/viewtopic.php?t=81510>
- Metadaten per Script abfragen: <https://homematic-forum.de/forum/viewtopic.php?t=82029>
- CCU: Zyklische Statusmeldungen von Geräten erklärt: <https://technikkram.net/blog/2020/04/24/ccu-zyklische-statusmeldungen-von-geraeten-erklaert/>
- HmIP-ESI-GAS Installations- und Bedienungsanleitung (eQ-3): <https://homematic-ip.com/sites/default/files/downloads/hmip-esi-gas-um-web.pdf>
- HmIP-ESI IEC: Intervalle von 6 auf 3 Min. reduzieren: <https://homematic-forum.de/forum/viewtopic.php?t=85378>